

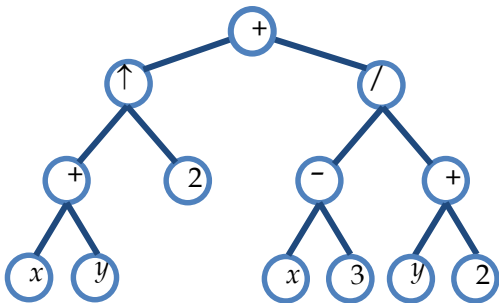
Trees

Representing Arithmetic Expressions

- Complicated arithmetic expressions can be represented by an ordered rooted tree
 - Internal vertices represent operators
 - Leaves represent operands
- Build the tree bottom-up
- Construct smaller subtrees
- Incorporate the smaller subtrees as part of larger subtrees

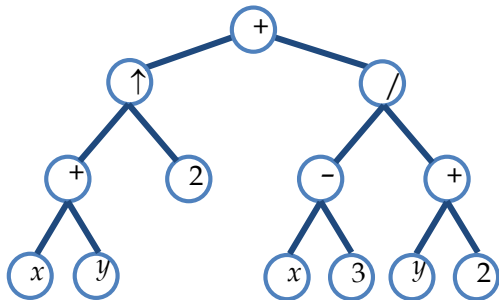
Example

$$(x+y)^2 + (x-3)/(y+2)$$



Infix Notation

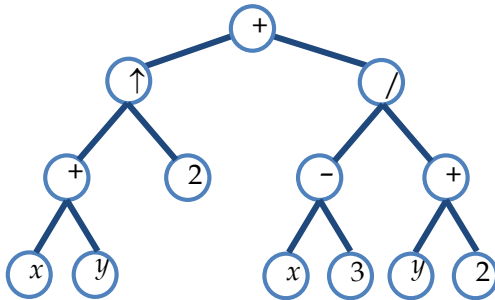
- Traverse in inorder adding parentheses for each operation



$$(((x + y) \uparrow 2) + ((x - 3) / (y + 2)))$$

Prefix Notation(Polish Notation)

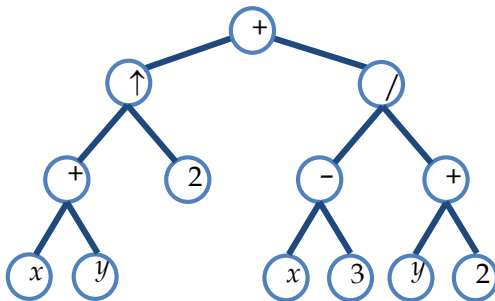
- Traverse in preorder



+ ↑ + x y 2 / - x 3 + y 2

Postfix Notation(Reverse Polish)

- Traverse in postorder

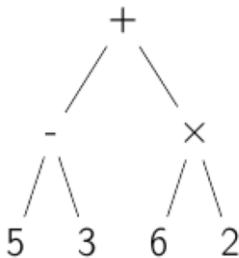


$x \ y \ + \ 2 \ \uparrow \ x \ 3 \ - \ y \ 2 \ + \ / \ +$

Question

If we have an expression tree and we want to evaluate that tree in order to compute the value of the expression, we want to evaluate the children before evaluating the arithmetic operator at an internal node. What traversal method does this mean we should use?

Sample expression tree representing $(5-3)+(6\times 2)(5-3)+(6\times 2)$:



- ☐ Post-order
- ☐ Breadth-first
- ☐ In-order
- ☐ Pre-order

Breadth-first

LevelTraversal(*tree*)

if *tree* = *nil*: return

Queue *q*

q.Enqueue(*tree*)

Breadth-first

LevelTraversal(*tree*)

```
if tree = nil:    return
```

```
Queue q
```

```
q.Enqueue(tree)
```

```
while not q.Empty():
```

```
    node ← q.Dequeue()
```

Breadth-first

LevelTraversal(*tree*)

```
if tree = nil:    return  
  
Queue q  
q.Enqueue(tree)  
while not q.Empty():  
    node ← q.Dequeue()  
    Print(node)
```

Breadth-first

LevelTraversal(*tree*)

```
if tree = nil : return
```

```
Queue q
```

```
q.Enqueue(tree)
```

```
while not q.Empty():
```

```
    node ← q.Dequeue()
```

```
    Print(node)
```

```
    if node.left ≠ nil :
```

```
        q.Enqueue(node.left)
```

Breadth-first

LevelTraversal(*tree*)

```
if tree = nil : return
```

```
Queue q
```

```
q.Enqueue(tree)
```

```
while not q.Empty():
```

```
    node ← q.Dequeue()
```

```
    Print(node)
```

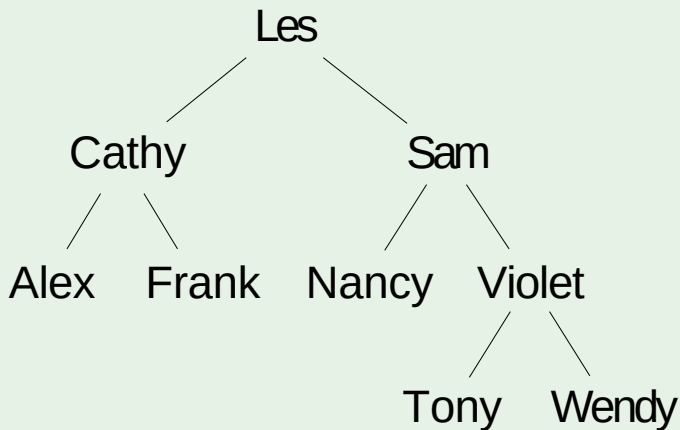
```
    if node.left ≠ nil :
```

```
        q.Enqueue(node.left)
```

```
    if node.right ≠ nil :
```

```
        q.Enqueue(node.right)
```

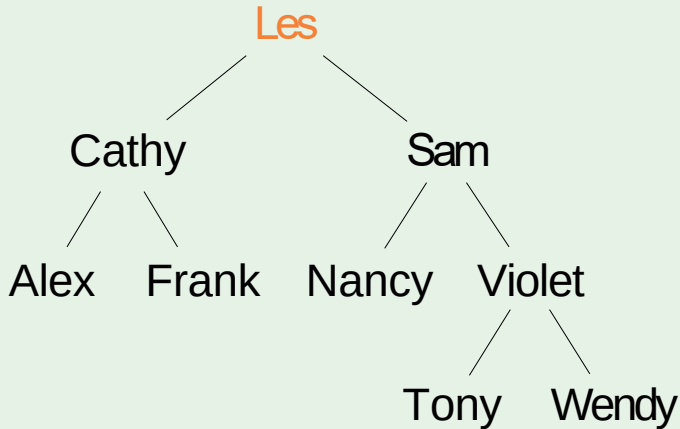
LevelTraversal



Output:

Queue: Les

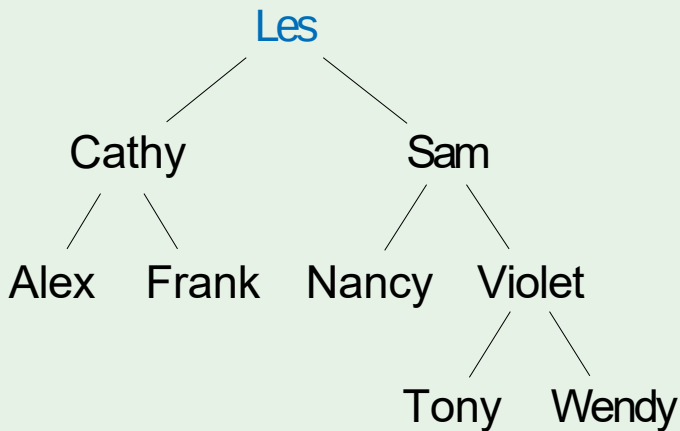
LevelTraversal



Output:

Queue:

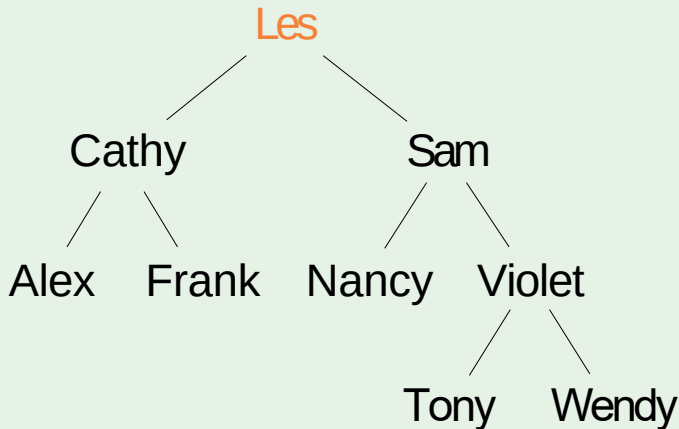
LevelTraversal



Output: Les

Queue:

LevelTraversal

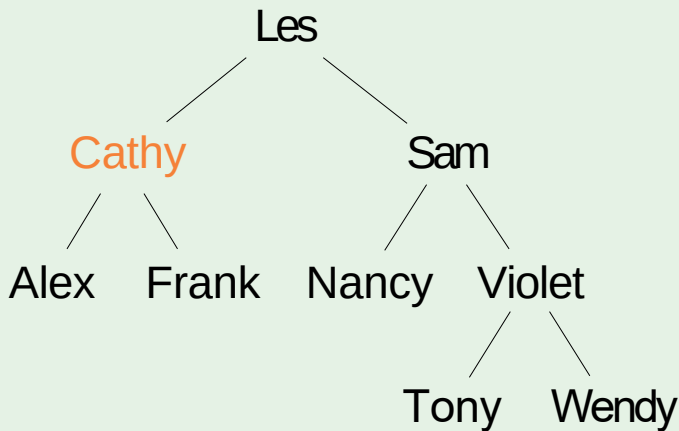


Output:

Les

Queue: Cathy, Sam

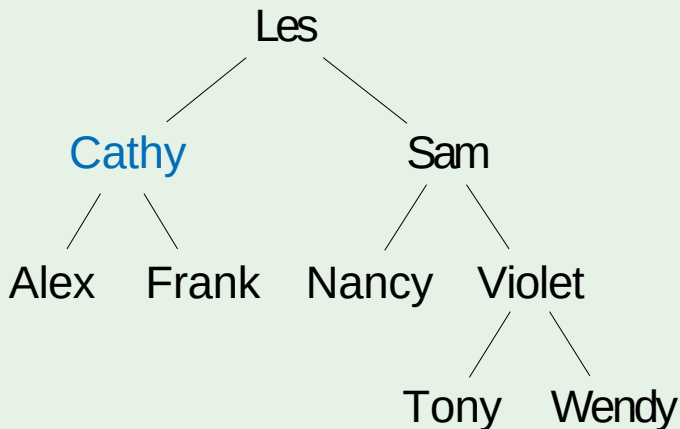
LevelTraversal



Output: Les

Queue: Sam

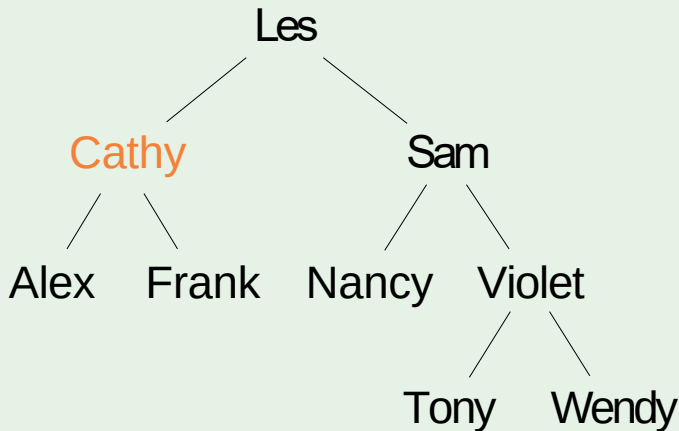
LevelTraversal



Output: Les Cathy

Queue: Sam

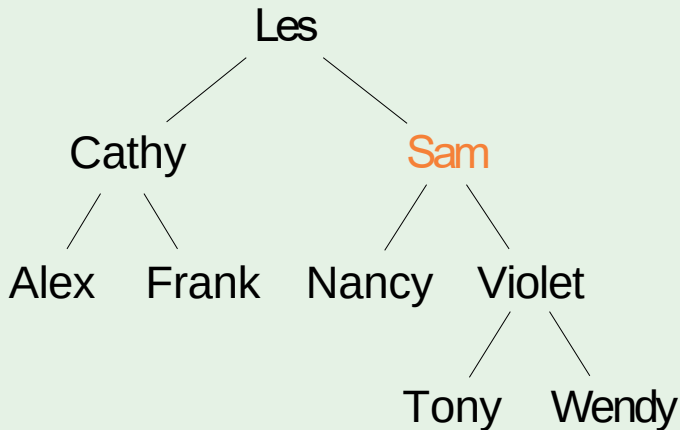
LevelTraversal



Output: Les Cathy

Queue: Sam,Alex, Frank

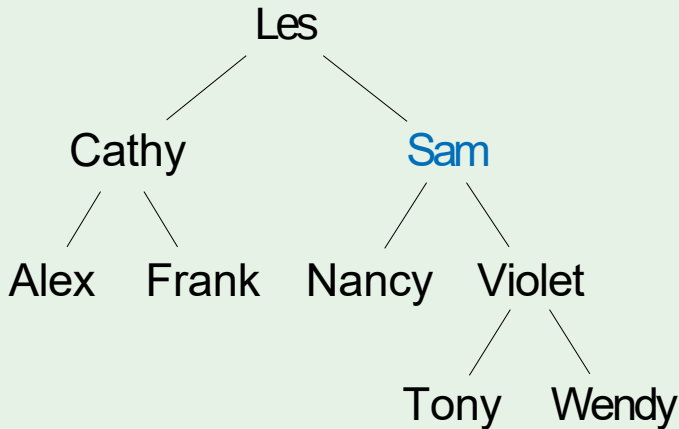
LevelTraversal



Output: Les Cathy

Queue: Alex, Frank

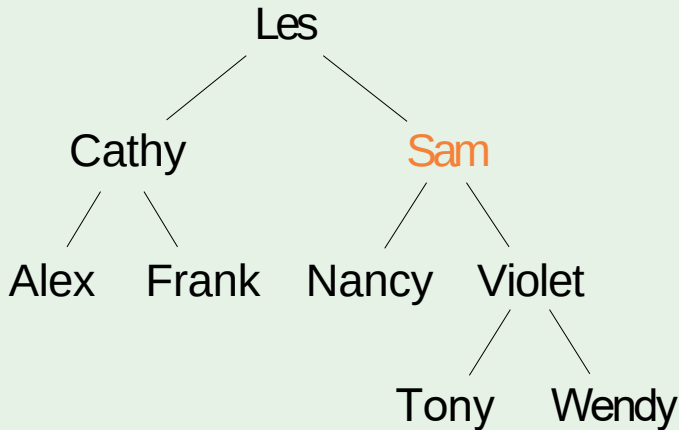
LevelTraversal



Output: Les Cathy Sam

Queue: Alex, Frank

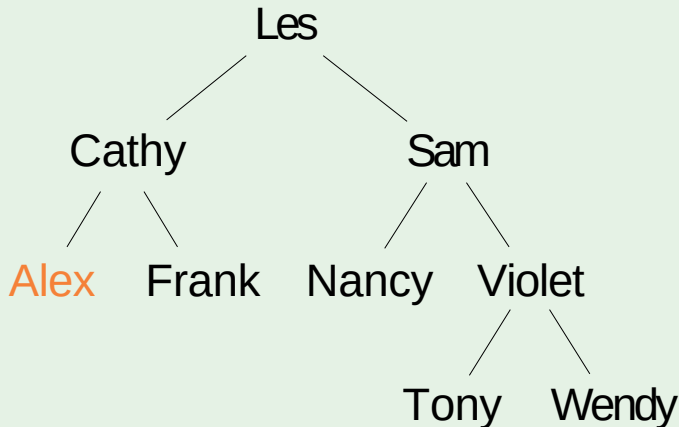
LevelTraversal



Output: Les Cathy
Sam

Queue: Alex, Frank, Nancy, Violet

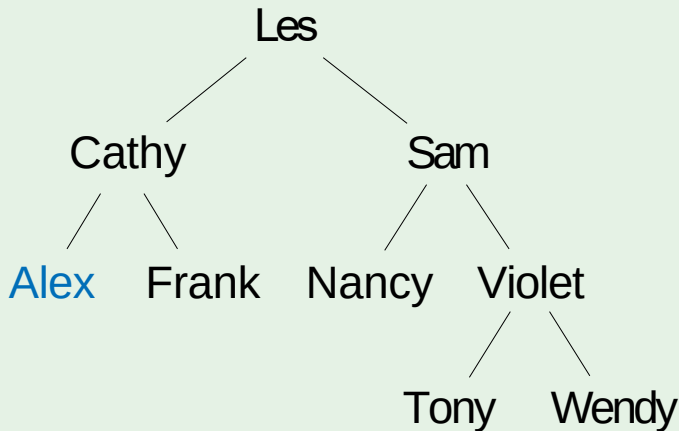
LevelTraversal



Output: Les Cathy Sam

Queue: Frank, Nancy, Violet

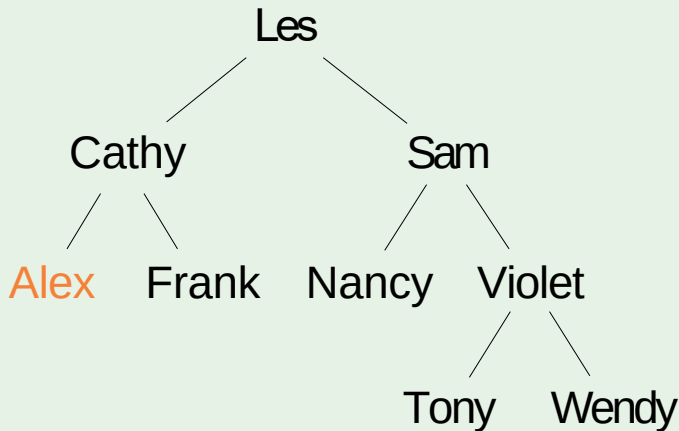
LevelTraversal



Output: Les Cathy Sam Alex

Queue: Frank, Nancy, Violet

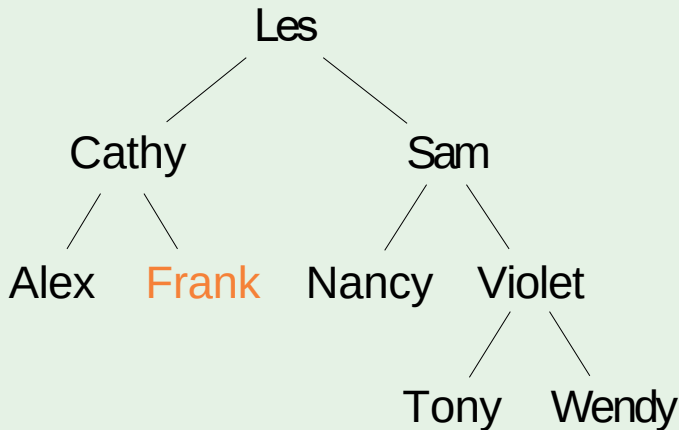
LevelTraversal



Output: Les Cathy Sam Alex

Queue: Frank, Nancy,

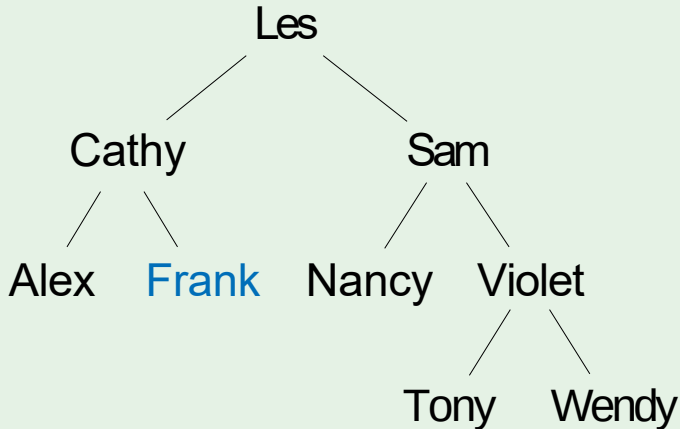
LevelTraversal



Output: Les Cathy Sam Alex

Queue: Nancy, Violet

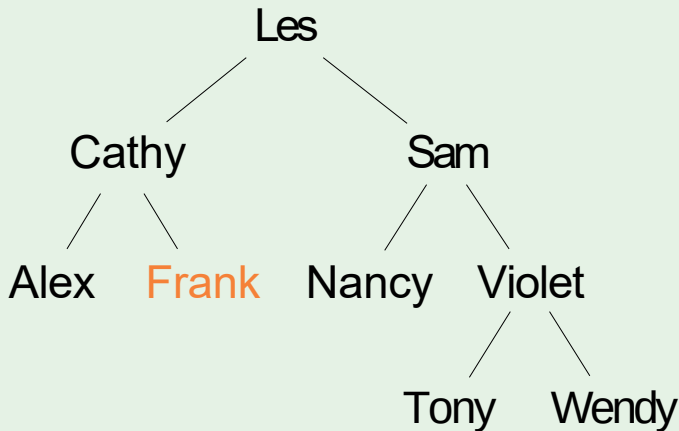
LevelTraversal



Output: Les Cathy Sam Alex Frank

Queue: Nancy, Violet

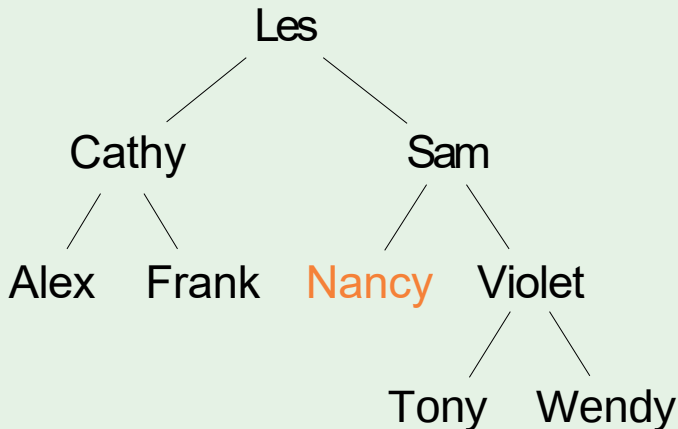
LevelTraversal



Output: Les Cathy Sam Alex Frank

Queue: Nancy, Violet

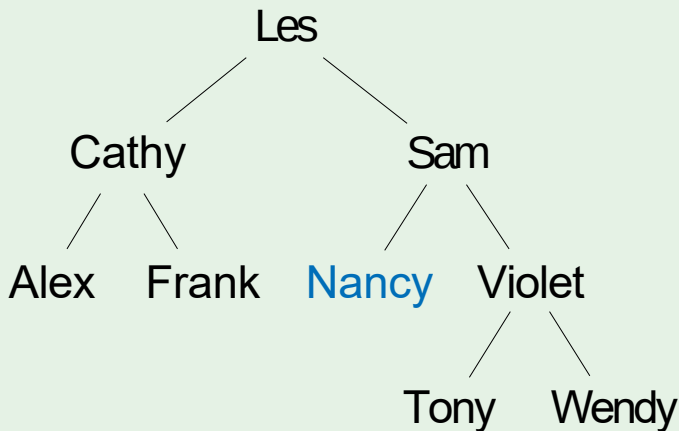
LevelTraversal



Output: Les Cathy Sam Alex Frank

Queue: Violet

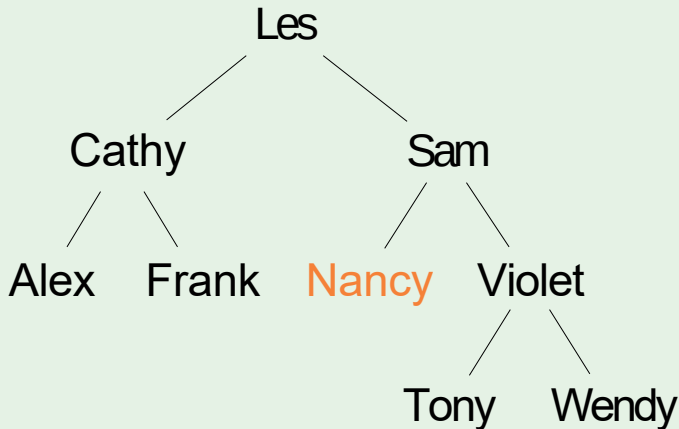
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy

Queue: Violet

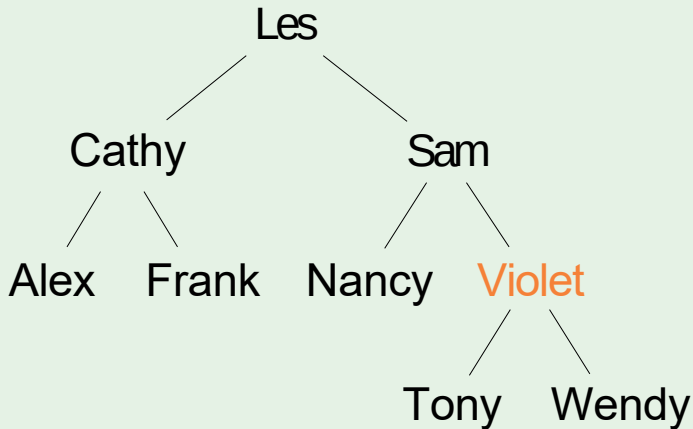
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy

Queue: Violet

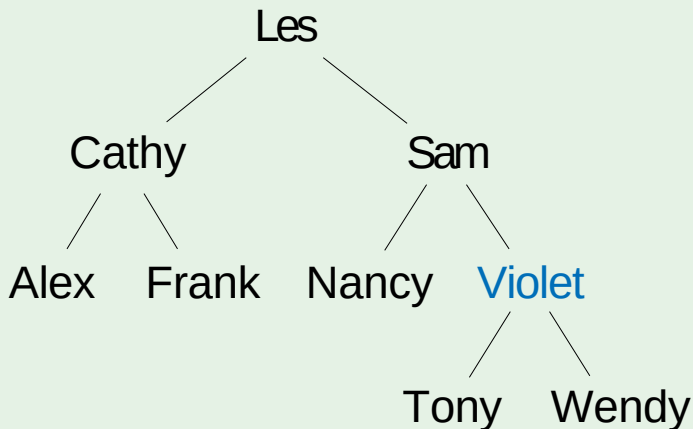
LevelTraversal



Output: Les Cathy Sam Alex Frank
Nancy

Queue:

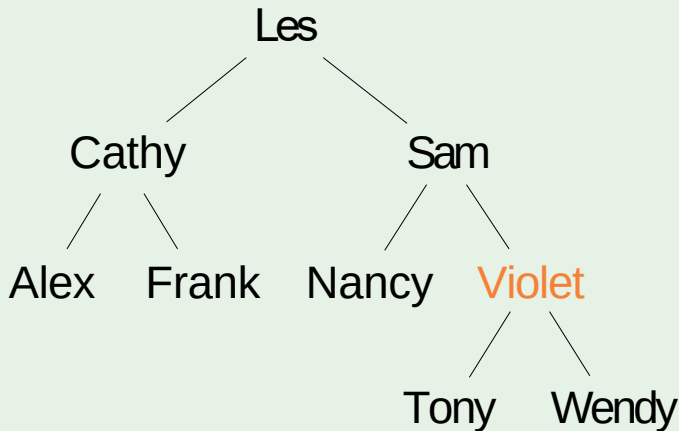
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy
Violet

Queue:

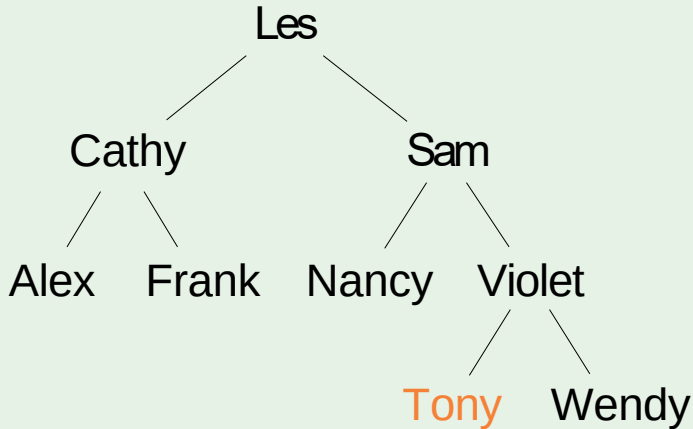
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy
Violet

Queue: Tony Wendy

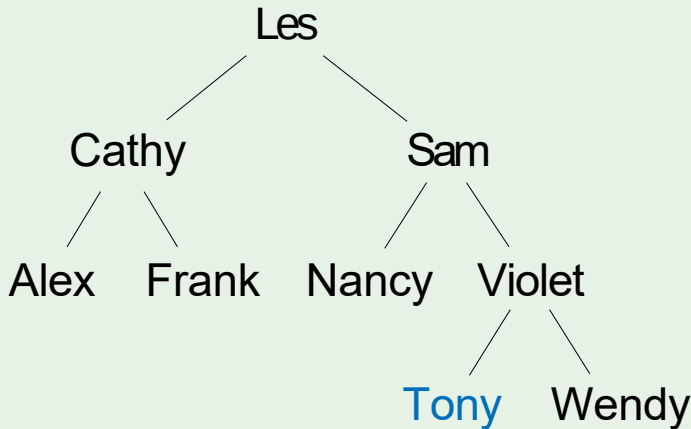
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy
Violet

Queue: Wendy

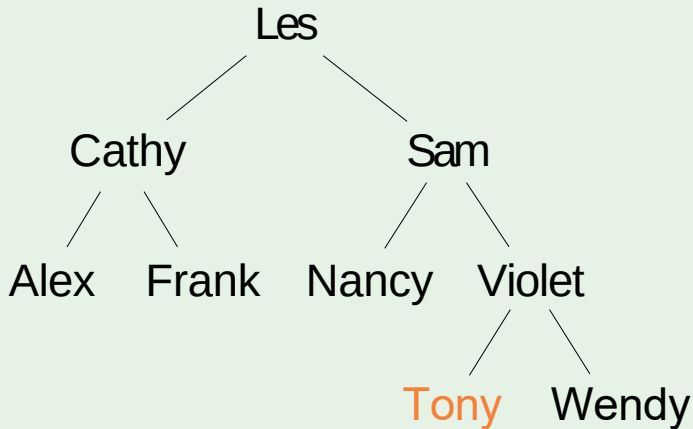
LevelTraversal



Output: Les Cathy Sam Alex Frank
Nancy Violet Tony

Queue: Wendy

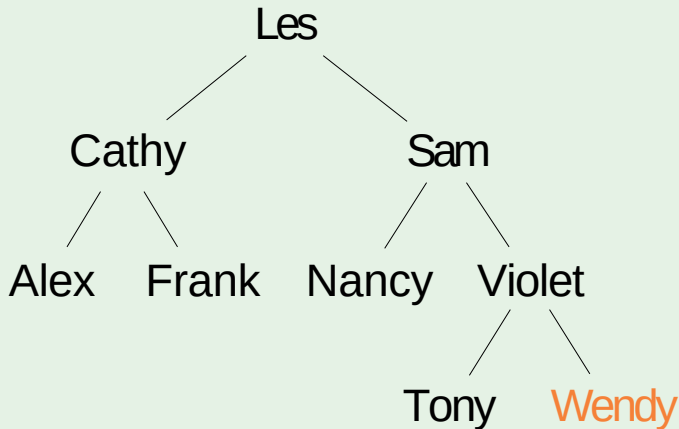
LevelTraversal



Output: Les Cathy Sam Alex Frank
Nancy Violet Tony

Queue: Wendy

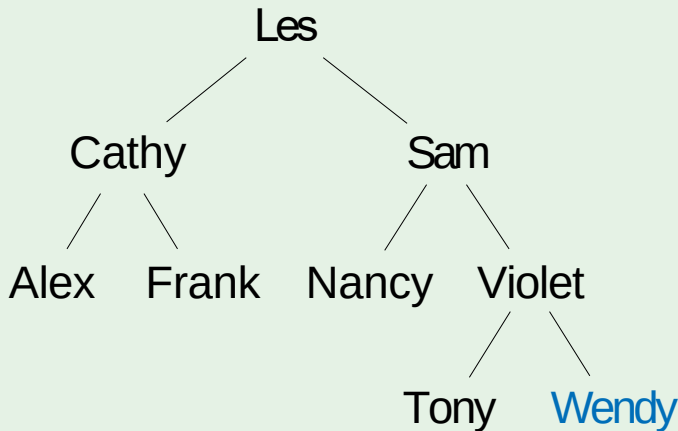
LevelTraversal



Output: Les Cathy Sam Alex Frank
Nancy Violet Tony

Queue:

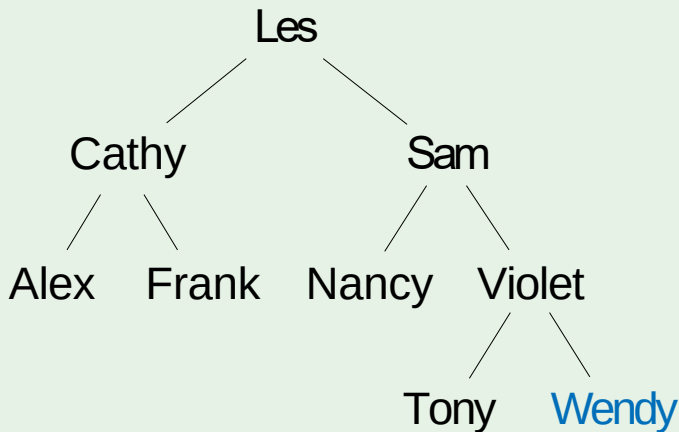
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy
Violet Tony Wendy

Queue:

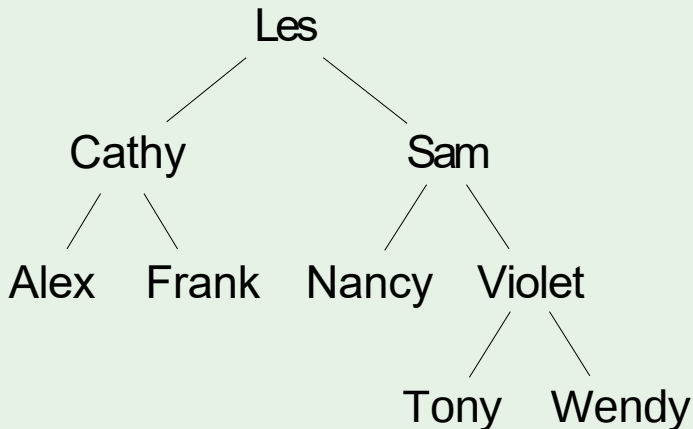
LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy
Violet Tony Wendy

Queue:

LevelTraversal



Output: Les Cathy Sam Alex Frank Nancy
Violet Tony Wendy

Queue:

Question 1

LevelOrderTraversal(tree):

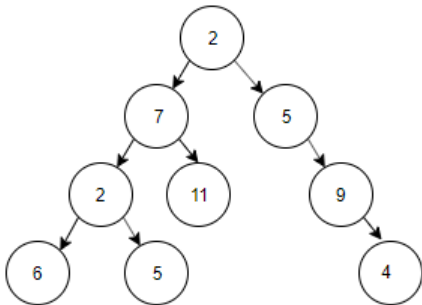
```
if tree == nil:  
    return
```

```
Queue q  
q.Enqueue(tree)
```

```
while not q.Empty():  
    node = q.Dequeue()  
    Print(node.value)
```

```
if node.left != nil:  
    q.Enqueue(node.left)
```

```
if node.right != nil:  
    q.Enqueue(node.right)
```



1.implement the level-order traversal (breadth-first traversal) using the pseudocode provided above. Your function should print the nodes of the tree in level-order.